



Odoo Day 4 – Odoo Architecture (Three-Tier, MVC, Request Flow, ORM)

This is one of the **most important topics** in Odoo Development. Every Odoo developer should understand it before creating custom modules.

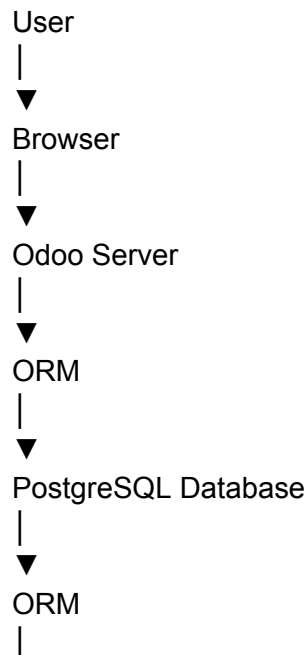
What is Odoo Architecture?

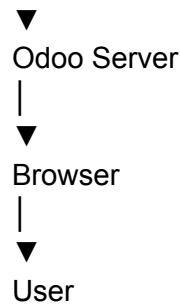
Odoo Architecture defines how data moves from the user to the database and back to the user.

Whenever you:

- Login
- Create a Customer
- Create a Product
- Confirm a Sales Order
- Create an Invoice

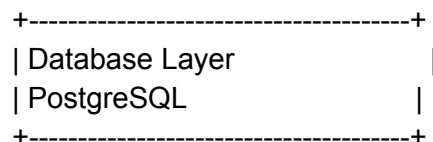
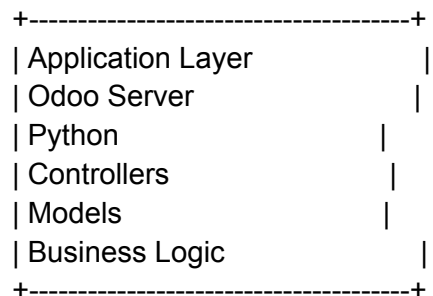
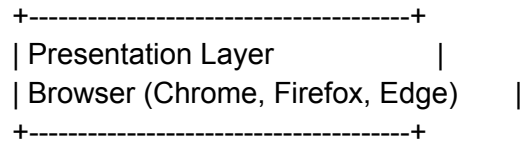
Odoo follows the same architecture.





Three-Tier Architecture

Odoo follows a **Three-Tier Architecture**.



Layer 1 – Presentation Layer

This is the **User Interface (UI)**.

Users interact with Odoo through a web browser.

Examples:

- Login Page
- Dashboard
- Sales Module
- CRM
- Inventory
- Forms
- Buttons
- Menus

Technology used:

- HTML
- CSS
- JavaScript
- XML Views

Example:

Chrome Browser



Sales Module



Click Save

Layer 2 – Application Layer

This is the **brain of Odoo**.

It processes all business logic.

Contains:

- Python

- Controllers
- Models
- Security
- ORM
- Business Rules

Example:

```
if amount > 10000:  
    approval_required = True
```

When you click **Save**, the Odoo server validates the data before storing it.

Layer 3 – Database Layer

The Database Layer stores all business information.

Database used by Odoo:

PostgreSQL

Stores:

- Customers
- Products
- Employees
- Sales Orders
- Purchase Orders
- Attendance
- Invoices
- Users
- Permissions

Example:

ID	Customer
1	ABC Company
2	Sugumar

Three-Tier Flow

User



Chrome Browser



Odoo Server



PostgreSQL



Data Saved

MVC Architecture

Odoo follows the **MVC (Model-View-Controller)** design pattern.

User



View (XML)



Controller (Python)



Model (Python)

|



ORM

|



PostgreSQL

M – Model

The **Model** represents the data and business logic.

Written in:

Python

Example:

```
from odoo import models, fields
```

```
class Student(models.Model):  
    _name = "student.student"
```

```
    name = fields.Char()  
    age = fields.Integer()
```

Model Responsibilities:

- Store data
- Validate data
- Apply business rules

Examples:

- Customer
- Product

- Student
 - Employee
-

V – View

The **View** is the user interface.

Written in:

XML

Examples:

- Form View
- List View (Tree)
- Kanban View
- Calendar View
- Search View

Example:

```
<form>
  <sheet>
    <group>
      <field name="name"/>
      <field name="age"/>
    </group>
  </sheet>
</form>
```

C – Controller

The **Controller** handles HTTP requests between the browser and the model.

Written in:

```
from odoo import http
```

```
class Student(http.Controller):
```

```
@http.route('/student')
def student(self):
    return "Hello Student"
```

Responsibilities:

- Receive requests
 - Call models
 - Return responses
-

MVC Example

Suppose the user creates a new Student.

User

↓

Student Form (View)

↓

Controller

↓

Student Model

↓

ORM

↓

PostgreSQL

↓

Student Saved

↓

What is ORM?

ORM = Object Relational Mapping

ORM is a bridge between **Python** and **PostgreSQL**.

Instead of writing SQL, developers use Python methods.

Without ORM:

```
SELECT * FROM res_partner;
```

With ORM:

```
partners = self.env['res.partner'].search([])
```

ORM automatically converts Python into SQL.

ORM Flow

Python Code

↓

ORM

↓

SQL Query

↓

PostgreSQL

CRUD Operations Using ORM

Create

```
self.env['res.partner'].create({  
    'name': 'Sugumar'  
})
```

Equivalent SQL:

```
INSERT INTO res_partner (name)  
VALUES ('Sugumar');
```

Read

```
partners = self.env['res.partner'].search([])
```

Equivalent SQL:

```
SELECT * FROM res_partner;
```

Update

```
partner.write({  
    'name': 'Sugumar R'  
})
```

Equivalent SQL:

```
UPDATE res_partner  
SET name='Sugumar R';
```

Delete

```
partner.unlink()
```

Equivalent SQL:

```
DELETE FROM res_partner;
```

Complete Request Flow

Suppose a user creates a customer.

User



Browser



HTTP Request



Odoo Controller



Python Model



ORM



PostgreSQL



ORM



Model



Controller



XML View

↓

Browser

↓

Customer Created Successfully

Odoo Internal Components

Browser

↓

HTTP Request

↓

Odoo Server

↓

Authentication

↓

Security

↓

Controller

↓

Model

↓

ORM

↓

Real Example

You enter:

Customer Name : Sugumar

Click:

Save

Flow:

Browser



Controller



Model



ORM



res_partner Table



Database



Saved Successfully

Advantages of Odoo Architecture

-  Clear separation of UI, logic, and data.
 -  Modular and scalable.
 -  Easy to maintain.
 -  Secure through ORM and access control.
 -  Reusable components.
 -  Fast development with Python and ORM.
-

Real-Life Analogy

Imagine a bank.

Customer



Reception Counter (Browser)



Bank Officer (Controller)



Bank Rules (Model)



Clerk (ORM)



Vault (PostgreSQL)



Money Stored



Receipt Returned

Mapping to Odoo:

Bank Example	Odoo
Customer	User
Reception	Browser
Bank Officer	Controller
Bank Rules	Model
Clerk	ORM
Vault	PostgreSQL Database

Interview Questions

1. What architecture does Odoo use?

Answer: Odoo follows **Three-Tier Architecture** and the **MVC (Model-View-Controller)** design pattern.

2. Which database does Odoo use?

Answer: PostgreSQL.

3. What is ORM?

Answer: ORM (Object Relational Mapping) is a layer that allows developers to interact with the PostgreSQL database using Python code instead of writing SQL queries directly.

4. Which language is used for Models?

Answer: Python.

5. Which language is used for Views?

Answer: XML.

6. What is the role of the Controller?

Answer: It receives HTTP requests from the browser, calls the appropriate model, and returns the response to the user interface.

Day 4 Summary

Topic	Description
Three-Tier Architecture	Presentation Layer, Application Layer, Database Layer
MVC	Model, View, Controller
ORM	Converts Python operations into SQL queries
Request Flow	Browser → Controller → Model → ORM → PostgreSQL → Browser
Database	PostgreSQL stores all Odoo business data
Models	Python classes representing business objects
Views	XML files defining the user interface
Controllers	Handle web requests and responses

Key Point to Remember

Every action in Odoo follows the same path:

User → Browser → Odoo Server → Model → ORM → PostgreSQL → ORM → Odoo Server → Browser → User

Once you understand this flow, learning module development in the next lessons becomes much easier.